



Praktikum Pemrograman Mobile

Pertemuan 2:

Material Design 3: Components & Forms

Disusun oleh :

Azis Amirulbahar, S.P.d., M.T.I.

Program Studi Informatika

Fakultas Teknik

Universitas Jenderal Soedirman

2026



DAFTAR ISI

DAFTAR ISI	2
PENDAHULUAN	3
A. Pengantar	3
B. Tujuan Praktikum	4
C. Alat dan Bahan	5
D. Dasar Teori.....	5
TAHAPAN PRAKTIKUM	6
A. Modifikasi Color.kt pada Project Android	6
B. Modifikasi Theme (Tema Pada Project Android)	7
C. Modifikasi Tipografi	9
D. Membuat Screen Baru	13
E. Modifikasi Main Activity dan Membuat Screen Baru	26
F. Membuat Halaman Navigasi	30
G. Membuat SnackBar	31
H. Memahami Dynamic Color	33

PENDAHULUAN

A. Pengantar

Theme adalah sekumpulan aturan visual yang menentukan bagaimana elemen-elemen antarmuka aplikasi ditampilkan, termasuk warna, tipografi, dan bentuk. Dengan menggunakan Theme, pengembang tidak perlu menentukan warna, ukuran teks, atau bentuk komponen secara berulang pada setiap halaman. **Dalam implementasi project berbasis Android menggunakan compose, theme menyediakan** mekanisme terpusat untuk konfigurasi visual yang terdiri dari **warna (color), tipografi (typography), dan bentuk (shapes)** seluruh hierarki antarmuka (UI). Theme pada Jetpack Compose memiliki karakteristik sebagai berikut:

1. **State-Driven & Reaktif:** Tema direpresentasikan sebagai struktur data Kotlin murni (ColorScheme, Typography, Shapes). Setiap perubahan nilai tema (misalnya beralih ke Mode Gelap atau Dynamic Color) akan otomatis memicu rekomposisi (*recomposition*) pada komponen yang membaca token terkait. **Recomposition** merupakan proses ketika **Jetpack Compose menjalankan kembali bagian kode UI yang diperlukan karena data atau keadaan (state) yang digunakan oleh UI tersebut berubah. Recomposition bukan berarti seluruh aplikasi dibuat ulang dari awal.**
2. **Implicit Dependency Injection:** Menggunakan CompositionLocalProvider di balik layar untuk menyuplai nilai token desain ke sub-pohon komponen tanpa perlu meneruskan parameter (*prop drilling*) secara manual.
3. **Role-Based Styling:** Memisahkan nilai mentah (seperti kode heksadesimal atau ukuran piksel) dari fungsi semantiknya di UI, sehingga komponen merujuk pada peran fungsional (misal: MaterialTheme.colorScheme.primary) alih-alih nilai statis.

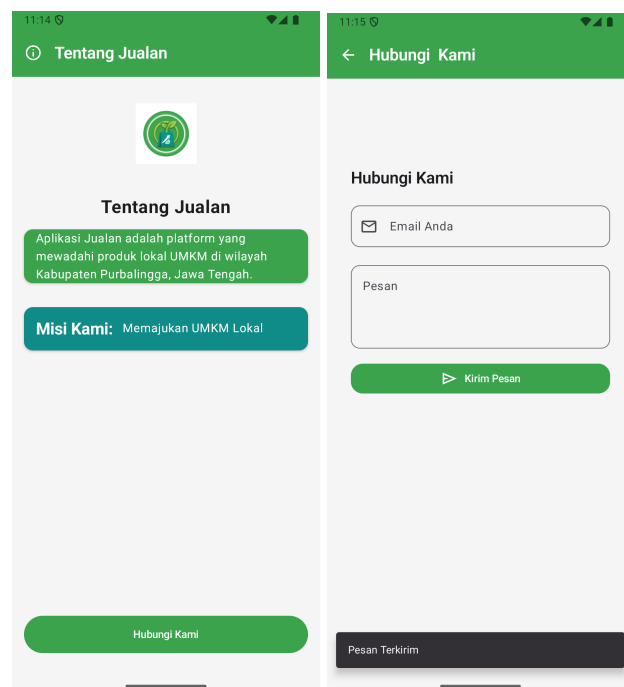
Material Design 3 (M3), atau yang sering dikenal dengan sebutan **Material You**, adalah iterasi evolusioner terbaru dari sistem desain (*design system*) sumber

terbuka ciptaan Google. M3 berfungsi sebagai panduan standar dan komprehensif untuk merancang antarmuka pengguna (UI) dan pengalaman pengguna (UX) modern, khususnya pada ekosistem Android.

B. Tujuan Praktikum

1. Mahasiswa mampu memahami dan menerapkan panduan visual Material Design 3 (MD3) menggunakan Jetpack Compose.
2. Mahasiswa mampu menggunakan komponen kerangka dasar layar seperti Scaffold dan TopAppBar.
3. Mahasiswa mampu merancang dan mengambil data dari formulir interaktif menggunakan komponen seperti TextField, Button, Checkbox, dan RadioButton.

Hasil akhir praktikum adalah sebagai berikut:



Gambar 1. Hasil Akhir Praktikum

C. Alat dan Bahan

1. Perangkat keras (PC/Laptop) dengan spesifikasi minimum sesuai standar Android Studio.
2. Android Studio (versi terbaru yang sudah mendukung Jetpack Compose secara bawaan).
3. Emulator Android (AVD) atau perangkat fisik Android yang terhubung melalui *debugging* (USB/Wi-Fi).

D. Dasar Teori

1. **Material Design 3 (Material You)/MD3** adalah sistem desain terbaru dari Google yang mengedepankan personalisasi, aksesibilitas, dan tata letak yang adaptif. Pada Jetpack Compose, MD3 diimplementasikan melalui pustaka bawaan `androidx.compose.material3`. Dengan MD3, komponen seperti tombol dan kartu otomatis memiliki sudut yang lebih membulat (*rounded*) dan warna yang menyesuaikan dengan tema sistem. Untuk mempelajari lebih dalam tentang MD3 silakan kunjungi tautan berikut: <https://m3.material.io/>
2. **Scaffold** adalah komponen fondasi yang menyediakan kerangka tata letak visual tingkat atas. Scaffold secara otomatis mengatur tempat untuk komponen-komponen standar aplikasi, seperti palang aplikasi atas (*Top App Bar*), tombol melayang (*Floating Action Button*), dan navigasi bawah (*Bottom Navigation*), sehingga isi halaman Anda tidak akan tertimpa oleh komponen-komponen tersebut.
3. **Snackbar** adalah komponen antarmuka pengguna (UI) pada Android yang berfungsi untuk memberikan umpan balik (*feedback*) ringan kepada pengguna mengenai suatu operasi atau status aplikasi. Komponen ini muncul sebagai bilah pesan kecil yang melayang di bagian bawah layar. Snackbar didesain bersifat *non-intrusif*, artinya kemunculannya tidak mengunci layar atau menghalangi pengguna untuk tetap berinteraksi dengan aplikasi. Pesan pada Snackbar akan menghilang secara otomatis setelah batas waktu tertentu habis, atau ketika pengguna melakukan interaksi lain yang relevan.

TAHAPAN PRAKTIKUM

Pada bagian ini, peserta diminta untuk mengikuti tahapan – tahapan untuk menghasilkan aplikasi sederhana seperti pada gambar 1 di halaman sebelumnya.

A. Modifikasi Color.kt pada Project Android

Dalam project Android Jetpack Compose, file **Color.kt** berfungsi sebagai tempat utama untuk mendefinisikan palet warna yang akan digunakan di seluruh aplikasi yang dibuat. File `Color.kt` dibuat secara otomatis saat membuat project Compose baru dan terletak di dalam package `ui.theme`. Jetpack Compose menggunakan class `androidx.compose.ui.graphics.Color` untuk membaca warna. Warna ditulis dalam format **Hexadecimal ARGB** (Alpha, Red, Green, Blue). Contoh:

```
val Purple80 = Color(0xFFD0BCFF)
```

Penjelasan:

- `0x`: Penanda bahwa nilai hexadecimal
- `FF`: Alpha channel (Tingkat transparansi/Opacity). `FF` berarti 100% solid (tidak transparan) sedangkan `00` berarti transparan total.
- `D0BCFF`: Kode warna .

Silakan ubah file `Color.kt` seperti berikut:

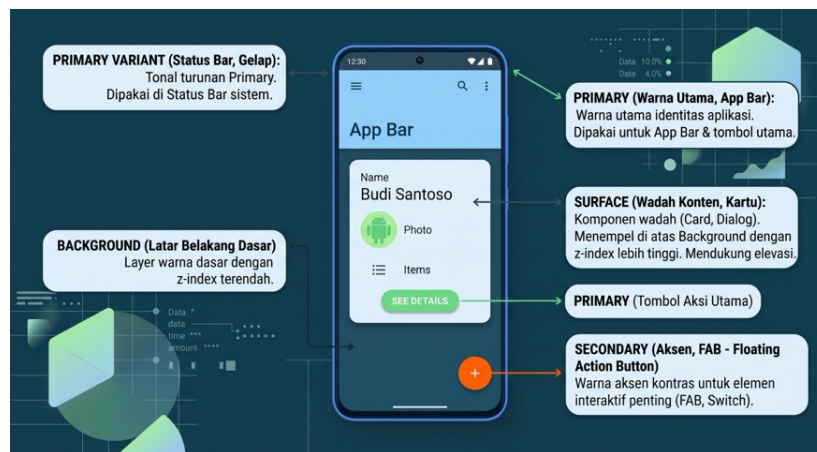
```
3 import androidx.compose.ui.graphics.Color
4
5 ■ val Primary = Color(0xFF3AA34B)
6 ■ val PrimaryVariant = Color(0xFF0F8B88)
7 ■ val Secondary = Color(0xFF76BA43)
8 ■ val Background = Color(0xFFFF5F5F)
9 ■ val Surface = Color(0xFFFFFFFF)
```

Berikut adalah daftar warna *hexadecimal* pada konfigurasi warna yang dilakukan:

Nama Variabel	Kode Hex
Primary	#3AA34B
PrimaryVariant	#0F8B88
Secondary	#76BA43
Background	#F5F5F5
Surface	#FFFFFF

B. Modifikasi Theme (Tema Pada Project Android)

Pada implementasi sebelumnya, `Color.kt` berfungsi sebagai tempat untuk mendefinisikan dan menyimpan nilai warna mentah (seperti kode heksadesimal) ke dalam variabel. Selanjutnya, `Theme.kt` akan memanggil warna-warna tersebut dan memetakannya ke dalam peran desain yang spesifik, seperti warna utama (*Primary*) atau warna latar belakang (*Background*).



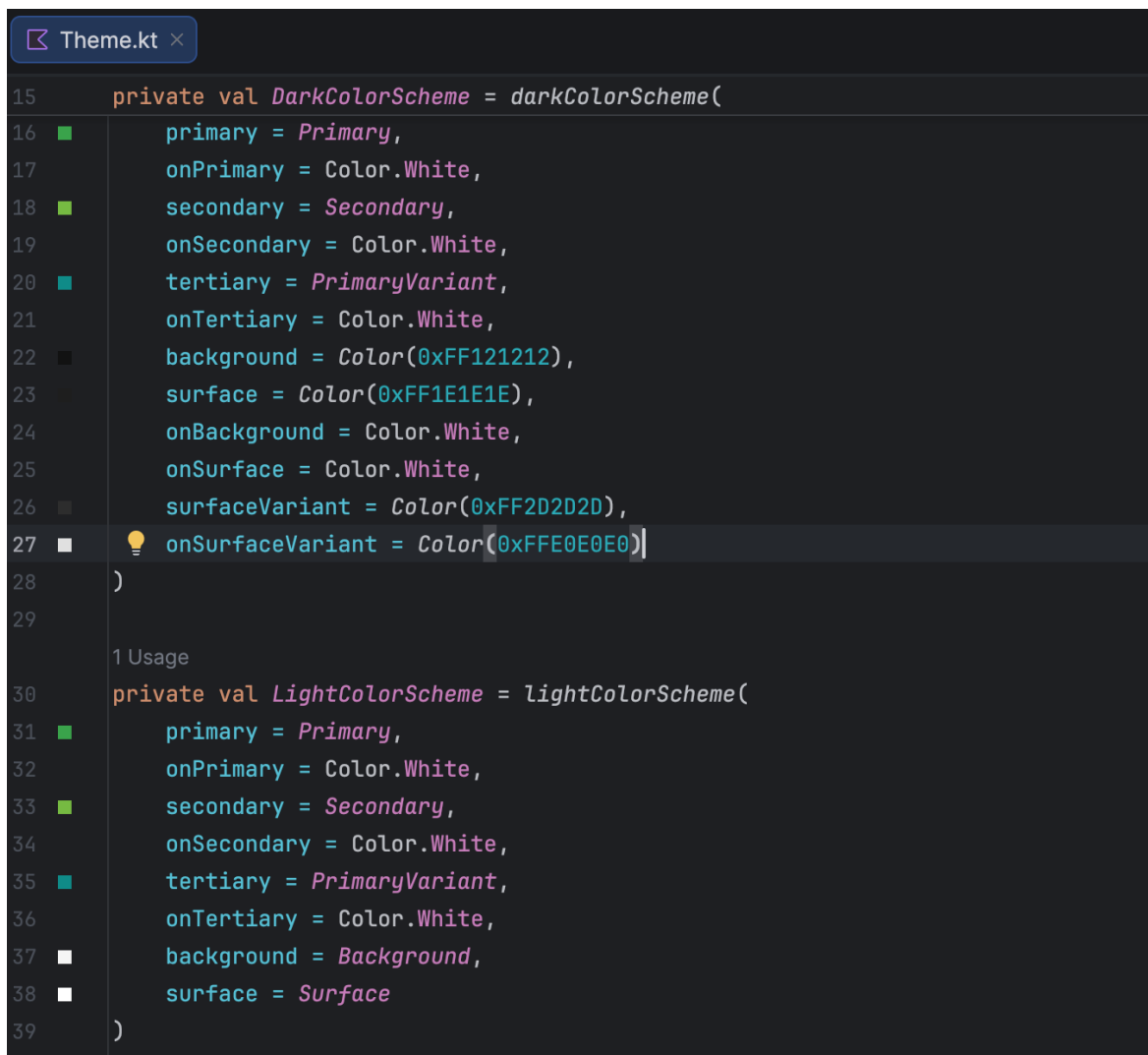
Gambar 2. Color Role pada Project Mobile berbasis Android

Dalam project Android Jetpack Compose, `Theme.kt` adalah file konfigurasi utama yang berfungsi untuk mendefinisikan dan menerapkan sistem desain (*design system*) antarmuka aplikasi secara terpusat menggunakan standar Material Design. Secara teknis, file ini memuat sebuah fungsi `Composable` yang mengelompokkan dan mengatur tiga elemen visual utama:

1. **Skema Warna (Color Scheme):** Memetakan warna-warna baku dari `Color.kt` ke dalam hierarki sistem (seperti *Primary*, *Secondary*, *Background*, *Surface*) dan menyediakan logika perpindahan otomatis antara mode terang (*Light Mode*) dan mode gelap (*Dark Mode*).
2. **Tipografi (Typography):** Menetapkan standar ukuran, jenis, dan ketebalan huruf untuk berbagai elemen teks (seperti judul, paragraf, dan label tombol).
3. **Bentuk (Shapes):** Menentukan gaya struktur geometris komponen antarmuka, seperti tingkat kelengkungan sudut pada tombol, kartu, atau menu.

Dengan menggunakan fungsi dari `Theme.kt` sebagai wadah utama (*wrapper*), setiap komponen antarmuka yang ada di dalam aplikasi akan mewarisi properti desain tersebut secara seragam, sehingga pengembang tidak perlu mendeklarasikan ulang warna atau gaya teks pada setiap layar secara manual.

Silakan ubah theme.kt sesuai kode berikut:



```
Theme.kt x
15 private val DarkColorScheme = darkColorScheme(
16     primary = Primary,
17     onPrimary = Color.White,
18     secondary = Secondary,
19     onSecondary = Color.White,
20     tertiary = PrimaryVariant,
21     onTertiary = Color.White,
22     background = Color(0xFF121212),
23     surface = Color(0xFF1E1E1E),
24     onBackground = Color.White,
25     onSurface = Color.White,
26     surfaceVariant = Color(0xFF2D2D2D),
27     onSurfaceVariant = Color(0xFFE0E0E0)
28 )
29
30 1 Usage
31 private val LightColorScheme = lightColorScheme(
32     primary = Primary,
33     onPrimary = Color.White,
34     secondary = Secondary,
35     onSecondary = Color.White,
36     tertiary = PrimaryVariant,
37     onTertiary = Color.White,
38     background = Background,
39     surface = Surface
40 )
```

Gambar 3.Kode Modifikasi Color.kt

C.Modifikasi Tipografi

Tipografi adalah cara mengatur tampilan tulisan dalam sebuah aplikasi Android. Tulisan merupakan bagian penting dalam aplikasi karena digunakan untuk menyampaikan informasi kepada pengguna, misalnya untuk judul, menu, tombol, dan penjelasan. Oleh karena itu, tulisan perlu dibuat agar mudah dibaca dan dipahami.

Dalam aplikasi Android, tipografi mencakup beberapa hal, seperti jenis huruf, ukuran tulisan, dan ketebalan tulisan. Misalnya, judul biasanya dibuat lebih besar agar mudah terlihat, sedangkan tulisan penjelasan dibuat dengan ukuran yang nyaman untuk dibaca. Tombol juga menggunakan tulisan yang jelas agar pengguna mengetahui fungsi tombol tersebut.

Pada Jetpack Compose, Android menyediakan pengaturan gaya tulisan agar tampilan aplikasi menjadi lebih rapi dan konsisten. Artinya, tulisan pada setiap bagian aplikasi dapat menggunakan aturan yang sama. Sebagai contoh, semua judul dapat memiliki ukuran yang seragam, sedangkan semua teks penjelasan menggunakan ukuran yang sesuai untuk isi.

Secara sederhana, tipografi dapat dibandingkan dengan sebuah buku. Judul buku dibuat besar agar mudah dikenali, judul bab dibuat berbeda dari isi, dan teks isi dibuat nyaman untuk dibaca. Hal yang sama juga diterapkan dalam aplikasi Android agar pengguna dapat dengan mudah memahami informasi yang ditampilkan.

Dengan penggunaan tipografi yang baik, aplikasi akan terlihat lebih rapi, nyaman digunakan, dan informasi di dalamnya menjadi lebih mudah dibaca oleh pengguna. Untuk mendalami teori tentang tipografi, silakan kunjungi tautan berikut: <https://m3.material.io/styles/typography>. Setelah kita membuka file `Type.kt`, kita akan melihat isi dari file tersebut adalah seperti pada gambar berikut:

```
val Typography = Typography(  
    bodyLarge = TextStyle(  
        fontFamily = FontFamily.Default,  
        fontWeight = FontWeight.Normal,  
        fontSize = 16.sp,  
        lineHeight = 24.sp,  
        letterSpacing = 0.5.sp  
    ),  
    /* Other default text styles to override  
    titleLarge = TextStyle(  
        fontFamily = FontFamily.Default,  
        fontWeight = FontWeight.Normal,  
        fontSize = 22.sp,  
        lineHeight = 28.sp,  
        letterSpacing = 0.sp  
    ),  
    labelSmall = TextStyle(  
        fontFamily = FontFamily.Default,  
        fontWeight = FontWeight.Medium,  
        fontSize = 11.sp,  
        lineHeight = 16.sp,  
        letterSpacing = 0.5.sp  
    )  
    */  
)
```

Gambar 4. Kode Bawaan dari Project Android Studio

Kode di atas merupakan kode yang dihasilkan secara otomatis oleh android studio.

Pada tahapan ini, kita akan belajar untuk memodifikasi tipografi. Untuk memulai melakukan kustom tipografi pada project Android yang sedang kita kembangkan, kita akan melakukan modifikasi kode pada **Type.kt**. Silakan hapus terlebih dahulu baris kode tersebut dan tersisa deklarasi tipografi sehingga seperti gambar 4 di bawah ini:

1 Usage

```
val Typography = Typography(  
  
)
```

Gambar 5. Menghapus Kode Bawaan dari Project Android Studio

Selanjutnya tuliskan kode berikut:

```
val Typography = Typography(  
    // Style untuk teks berukuran besar (seperti judul utama layar)  
    headlineMedium = TextStyle(  
        fontFamily = FontFamily.SansSerif,  
        fontWeight = FontWeight.Bold,  
        fontSize = 28.sp,  
        lineHeight = 36.sp,  
        letterSpacing = (-0.5).sp  
    ),  
    // Style untuk judul di dalam Card atau TopAppBar  
    titleLarge = TextStyle(  
        fontFamily = FontFamily.SansSerif,  
        fontWeight = FontWeight.SemiBold,  
        fontSize = 22.sp,  
        lineHeight = 28.sp,  
        letterSpacing = 0.sp  
    ),  
    // Style untuk teks paragraf utama  
    bodyLarge = TextStyle(  
        fontFamily = FontFamily.SansSerif,  
        fontWeight = FontWeight.Normal,  
        fontSize = 16.sp,  
        lineHeight = 24.sp,  
        letterSpacing = 0.5.sp  
    ),  
)
```

Gambar 6. Modifikasi kode untuk headline, title, body

Di bawah deklarasi body large, tambahkan kode berikut:

```
// Style untuk teks paragraf sekunder  
bodyMedium = TextStyle(  
    fontFamily = FontFamily.SansSerif,  
    fontWeight = FontWeight.Normal,  
    fontSize = 14.sp,  
    lineHeight = 20.sp,  
    letterSpacing = 0.25.sp  
)  
// Style untuk teks berukuran kecil seperti di dalam Button atau label input  
labelLarge = TextStyle(  
    fontFamily = FontFamily.SansSerif,  
    fontWeight = FontWeight.Medium,  
    fontSize = 14.sp,  
    lineHeight = 20.sp,  
    letterSpacing = 0.1.sp  
)
```

Gambar 7. Modifikasi kode untuk body dan label

Penjelasan kode:

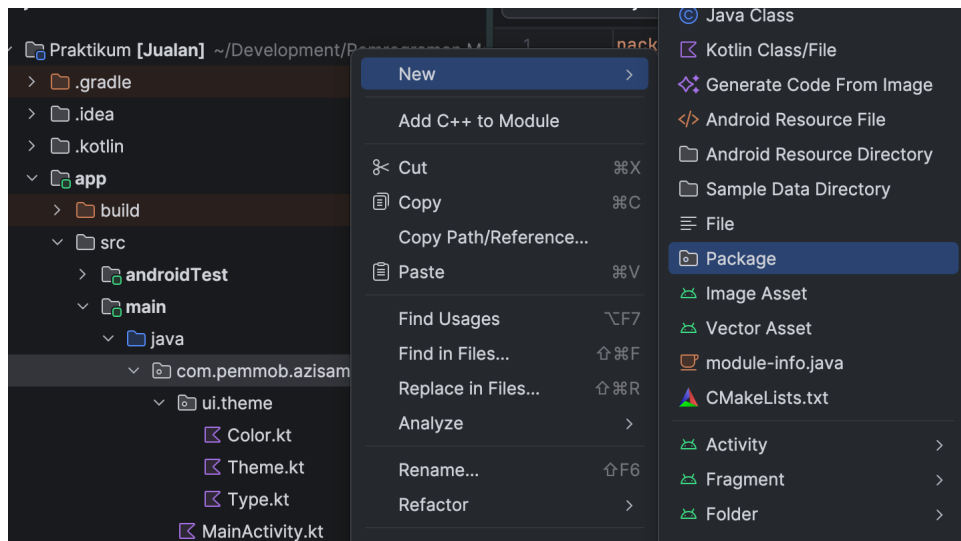
1. **fontFamily**: Jenis huruf/font yang digunakan. Di sini menggunakan `FontFamily.SansSerif` (huruf standar Android tanpa kait/sirip di ujung hurufnya).
2. **fontWeight**: Tingkat ketebalan huruf. Ada *Bold* (sangat tebal), *SemiBold* (agak tebal), *Medium* (sedang), dan *Normal* (biasa).
3. **fontSize**: Ukuran teks. Perhatikan bahwa ini menggunakan satuan **sp (Scaleable Pixels)**, bukan dp. Satuan sp membuat ukurannya dapat membesar atau mengecil secara otomatis mengikuti pengaturan ukuran huruf di sistem HP pengguna.
4. **lineHeight**: Jarak antar baris teks (seperti *line spacing* di Microsoft Word). Membuat teks paragraf tidak saling bertumpuk atas-bawah.
5. **letterSpacing**: Jarak renggang antar karakter/huruf. Terkadang judul besar butuh jarak huruf yang lebih rapat (contoh: `-0.5.sp`), sedangkan teks kecil butuh jarak lebih renggang agar mudah dibaca.

Seluruh variabel yang telah kita modifikasi, diakses oleh `JualanTheme` pada file `Theme.kt`. Kode `JualanTheme` berfungsi sebagai pengatur tampilan utama aplikasi Android yang dibuat menggunakan Jetpack Compose. Theme digunakan untuk menerapkan aturan tampilan secara konsisten pada seluruh bagian aplikasi. Pada kode tersebut, **Color** mengatur warna yang digunakan aplikasi dengan memilih `DarkColorScheme` ketika perangkat menggunakan Dark Mode atau `LightColorScheme` ketika menggunakan Light Mode. Sementara itu, **Typography** mengatur gaya tulisan, seperti ukuran dan bentuk teks yang digunakan dalam aplikasi. Semua pengaturan tersebut kemudian diterapkan melalui `MaterialTheme`, sehingga tampilan yang berada di dalam `JualanTheme` dapat menggunakan warna dan gaya tulisan yang sama secara konsisten.

D. Membuat Screen Baru

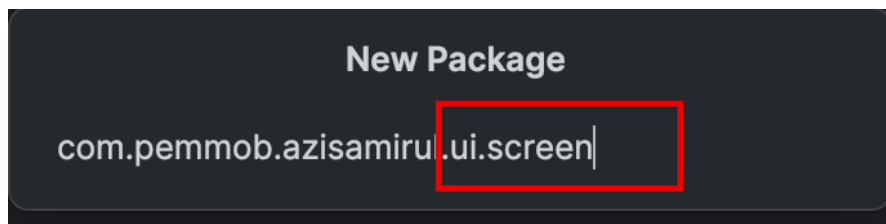
1. Membuat Package baru

Pada bagian ini, kita akan membuat package baru untuk mengelompokkan function khusus tampilan/UI. Silakan klik kanan pada bagian kemudian pilih **New → Package**:



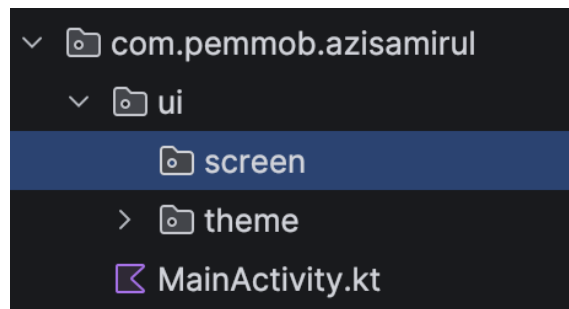
Gambar 8. Menu Membuat Package Baru

Kemudian masukkan **.ui.screen** dan tekan tombol **ENTER**.



Gambar 9. Mengisi nama package

Jika proses berhasil, maka terdapat package baru seperti pada gambar 10 berikut:

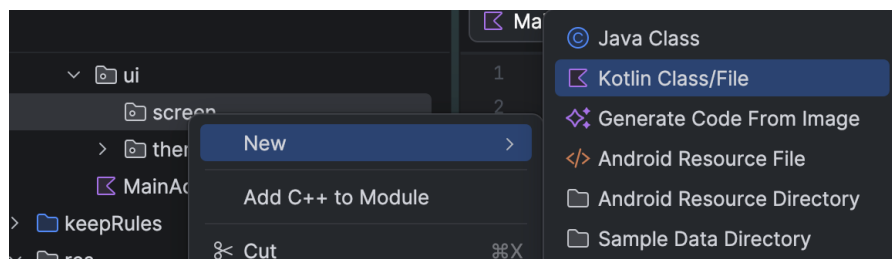


Gambar 10. Package Baru: ui.screen

4. Membuat function tanpa Deklarasi Class dengan Kotlin

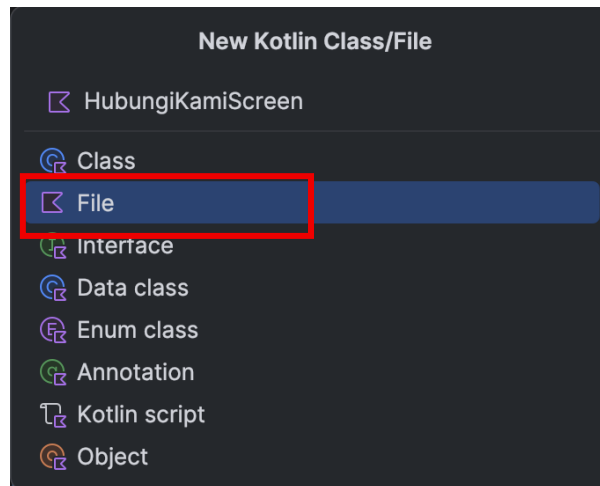
Pada praktik ini kita juga akan melakukan deklarasi function yang digunakan untuk mendeklarasikan *function* tentang screen yang berbentuk form. *Function* tanpa deklarasi Class disebut sebagai **Top-Level Functions**.

- a. Silakan buat File **.kt** baru pada package **ui.screen** seperti pada gambar berikut:



Gambar 11. Membuat File .kt pada package .ui.screen

b. Nama file menjadi: **HubungiKamiScreen** dengan tipe adalah **File**



5. Menambahkan *dependency* pada *build.gradle.kts*:

Dalam pengembangan aplikasi Android, kita tidak diwajibkan untuk membangun setiap fitur dari nol. Sering kali, kita membutuhkan *tools*, desain, atau potongan program yang sudah dibuat dengan baik oleh Google maupun komunitas pengembang lain, yang dikenal dengan istilah ***dependency*** (pustaka pendukung). Untuk memasukkan alat-alat tambahan ini ke dalam *project*, kita harus mendaftarkannya ke dalam sebuah file konfigurasi khusus yang bernama *build.gradle.kts*. Ketika pengembang menuliskan sebuah *dependency* ke dalamnya, berarti bahwa pengembang memberikan instruksi kepada program Android Studio untuk pergi ke internet, mengunduh fitur tersebut, dan memasangnya ke dalam proyek agar dapat langsung dipanggil saat menulis kode.

Pada pertemuan kali ini, kita akan mempraktikkan untuk menambahkan *dependency* berikut:

```
implementation(libs.androidx.navigation.runtime.ktx)
implementation(libs.androidx.navigation.compose)
```

Kode `libs.androidx.navigation.runtime.ktx` adalah sebuah "**Sistem Navigasi**" atau "**Pemandu Jalan**" resmi dari Google yang bertugas membangun

lorong dan pintu antar ruangan tersebut. Secara sederhana, kegunaan utamanya adalah sebagai berikut:

- a. **Mengatur Perpindahan Layar:** Alat ini membantu aplikasi Anda berpindah dari satu halaman ke halaman lainnya dengan mulus, tanpa Anda harus menulis kode yang rumit secara manual.
- b. **Mengatur Tombol "Kembali" (Back Button):** Ketika pengguna menekan tombol kembali di perangkat mereka, `libs.androidx.navigation.runtime.ktx` memastikan bahwa aplikasi kembali ke halaman sebelumnya dengan urutan yang benar, bukan malah keluar dari aplikasi secara tiba-tiba atau error.
- c. **Membawa Data Antar Layar:** melakukan pengiriman data dari satu halaman ke halaman lain. Misalnya, membawa data "Nama Pengguna" dari halaman Login untuk ditampilkan di halaman Beranda.

Sedangkan `implementation(libs.androidx.navigation.compose)` berfungsi untuk menghubungkan sistem navigasi halaman pada **Jetpack Compose**. Untuk melakukan penambahan dependency tersebut, silakan menuju ke: **build.gradle.kts** pada folder :app yang telah otomatis dibuat pada project android, kemudian pada blok dependencies, tambahkan baris kode berikut:

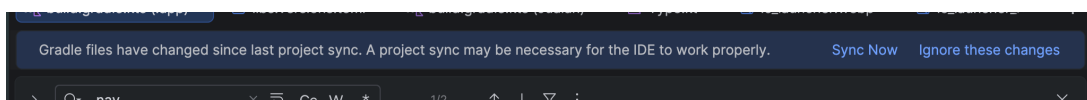
```
implementation(libs.androidx.navigation.runtime.ktx)
implementation(libs.androidx.navigation.compose)
```

```
dependencies {
    implementation(platform(dependencyProvider = libs.androidx.compose.bom))
    implementation(libs.androidx.activity.compose)
    implementation(libs.androidx.compose.material3)
    implementation(libs.androidx.compose.ui)
    implementation(libs.androidx.compose.ui.graphics)
    implementation(libs.androidx.compose.ui.tooling.preview)
    implementation(libs.androidx.core.ktx)
    implementation(libs.androidx.lifecycle.runtime.ktx)

    implementation(libs.androidx.navigation.compose)
    implementation(libs.androidx.navigation.runtime.ktx)
}
```

Gambar 12. Menambahkan Dependency Navigation pada build.gradle.kts

Setelah ditambahkan, klik tombol **"Sync Now"** yang terdapat pada bagian atas, atau pada menu **File → Sync Project with Gradle Files**.



Gambar 13. Sinkronisasi Gradle dengan Project

6. Membuat **Function** `HubungiKamiScreen()`:

Pada tahapan berikutnya, kita akan membuat function **HubungiKamiScreen** pada file **HubungiKamiScreen.kt**, seperti pada gambar 14 berikut:

```
@OptIn(markerClass = ExperimentalMaterial3Api::class)
@Composable
fun HubungiKamiScreen(navController: NavController) {

}
```

Unresolved reference 'NavController'
Import class 'NavController'

Gambar 14. Deklarasi Function `HubungiKamiScreen()`

Penjelasan kode:

- a. `@OptIn(ExperimentalMaterial3Api::class)`: **memberikan izin eksplisit terhadap penggunaan API (Application Programming Interface) yang membutuhkan persetujuan khusus.** Beberapa desain ini masih tergolong baru dan berlabel "eksperimental" (masih dalam tahap penyempurnaan).
- b. `@Composable`: label `@Composable` mengubah kode tersebut menjadi **sebuah elemen visual** (seperti tombol, gambar, atau layar utuh). Kode ini memberi tahu sistem bahwa bagian ini adalah desain yang harus digambar dan ditampilkan ke layar HP pengguna.
- c. `fun HubungiKamiScreen`: Kata `fun` adalah singkatan dari *function* (fungsi/kegiatan), sedangkan `HubungiKamiScreen` adalah **nama** yang kita sepakati untuk halaman ini.
- d. `(navController: NavController?)`: Deklarasi parameter `(navController: NavController?)` mendefinisikan bahwa fungsi tersebut menerima argumen bertipe `NavController` yang mengimplementasikan antarmuka *nullable*.
- e. **Membuat *Function* `HubungiKamiScreen()`:**

7. Menambahkan icon

Pada tahapan ini kita akan menambahkan icon untuk ditampilkan pada user interface (UI). Silakan unduh file berikut dengan format **.svg**:

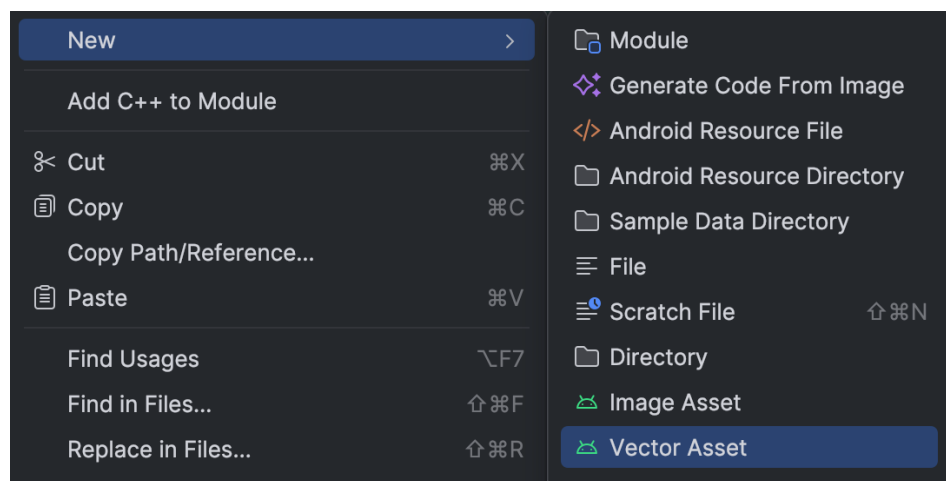
- a. https://fonts.google.com/icons?selected=Material+Symbols+Outlined:arrow_back:FILL@0;wght@400;GRAD@0;opsz@24&icon.size=24&icon.color=%231f1f1f
- b. <https://fonts.google.com/icons?selected=Material+Symbols+Outlined:mail:FILL@0;wght@400;GRAD@0;opsz@24&icon.size=24&icon.color=%231f1f1f&icon.query=mail>
- c. <https://fonts.google.com/icons?selected=Material+Symbols+Outlined:send:FILL@0;wght@400;GRAD@0;opsz@24&icon.size=24&icon.color=%231f1f1f&icon.query=send>

- d. <https://fonts.google.com/icons?selected=Material+Symbols+Outlined:info:FILL@0;wght@400;GRAD@0;opsz@24&icon.size=24&icon.color=%231f1f1f&icon.query=information>

Ubah masing – masing hasil *download* tersebut:

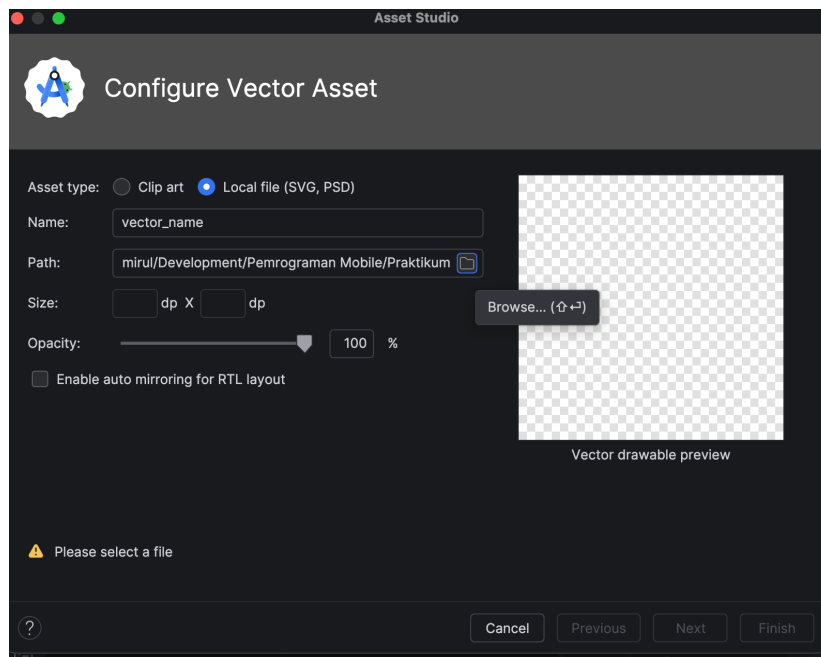
- a. back_icon.svg
- b. mail_icon.svg
- c. send_icon.svg
- d. info_icon.svg

Untuk melakukan import icon tersebut, silakan klik kanan pada folder app kemudian pilih New → Vector Asset.



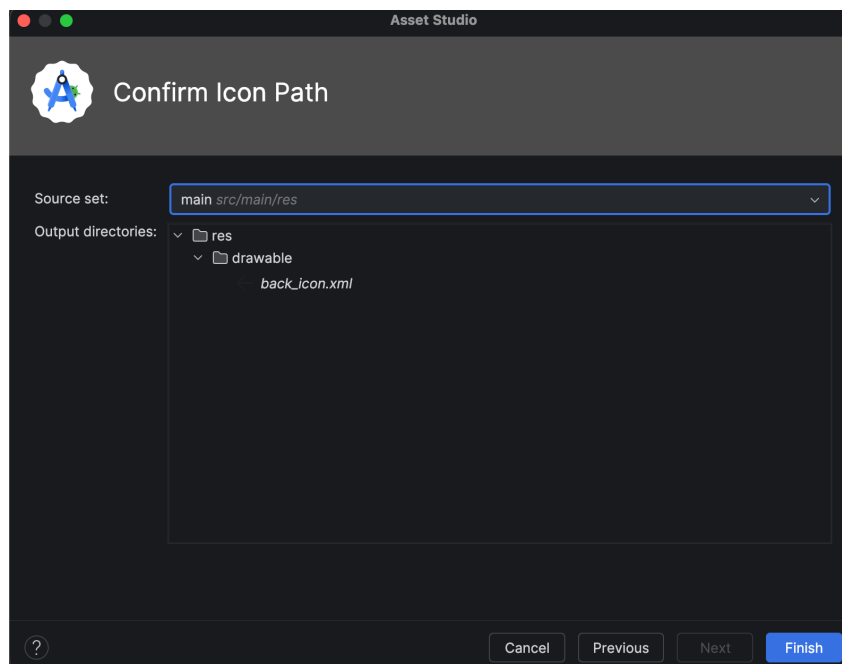
Gambar 15. Menu untuk menambahkan Icon

Kemudian pilih asset type: **Local File (SVG, PSD)** dan pada path pilih file yang telah diunduh pada tahapan sebelumnya.



Gambar 16. Konfigurasi Vektor yang Akan di-import

Kemudian klik **finish**. Gambar icon akan tersimpan pada folder: *res* → *drawable*



Gambar 17. Konfirmasi Import

8. Deklarasi Variabel

Pada tahapan ini, kita akan berlatih untuk melakukan deklarasi variabel.

Silakan tulis kode deklarasi variabel seperti pada gambar 15 berikut:

A screenshot of a code editor with a dark theme. It shows Kotlin code for a function named `HubungiKamiScreen`. The function takes a `NavController` parameter. Inside the function, two variables are declared: `emailText` and `messageText`. Both are declared using `remember` and `mutableStateOf` with an initial value of an empty string. The code is as follows:

```
@OptIn( ...markerClass = ExperimentalMaterial3Api::class)
@Composable
fun HubungiKamiScreen(navController: NavController) {
    var emailText by remember { mutableStateOf( value = "" ) }
    var messageText by remember { mutableStateOf( value = "" ) }
```

Gambar 18. Deklarasi Variabel

Penjelasan kode:

Kata kunci `mutableStateOf("")` menciptakan sebuah data khusus yang dapat memicu pembaruan layar secara otomatis; setiap kali teks di dalamnya berubah (misalnya saat pengguna mengetik), sistem akan merender ulang tampilan agar teks terbaru muncul di layar. Sementara itu, fungsi `remember` bertugas menjaga agar data yang sudah diketik pengguna tidak hilang atau terhapus ketika proses pembaruan layar tersebut terjadi. Penggunaan kata kunci `by` merupakan pintasan dari bahasa Kotlin agar kita bisa langsung membaca dan mengubah isi variabel tersebut selayaknya variabel biasa, tanpa harus menuliskan kode tambahan yang rumit di belakang layar. Pembaruan tampilan layar disebut dengan **rekomposisi** (*recomposition*), yaitu proses menggambar ulang fungsi komponen UI dari awal setiap kali ada data yang berubah. Ketika pengguna mengetik sesuatu, nilai di dalam `mutableStateOf` akan berubah dan secara otomatis memicu terjadinya rekomendasi agar huruf baru tersebut bisa langsung tampil di layar.

9. Membuat Scaffold

Tambahkan *Scaffold* di bawah deklarasi variabel.

```
Scaffold(  
  
){  
  
    paddingValues ->  
  
}
```

Gambar 19. Deklarasi Scaffold

10. Membuat Topbar

Dalam aplikasi Android, **Top Bar** (sering juga disebut *App Bar* atau *Header*) adalah bagian dari navigasi yang letaknya menempel di bagian paling atas layar ponsel. Selain menampilkan judul halaman, Top Bar juga sering dijadikan tempat untuk meletakkan tombol seperti "Kembali" ke halaman sebelumnya atau tombol menu pengaturan.

Melalui perintah `painterResource` yang terdapat dalam scope icon, kita memanggil gambar ikon yang sudah kita simpan di dalam proyek aplikasi. Tombol ini dilengkapi dengan pengaturan aksi pada bagian `onClick = { navController?.popBackStack() }` yaitu untuk kembali ke halaman sebelumnya.

```
Scaffold(  
    topBar = {  
        TopAppBar(  
            title = { Text("Hubungi Kami") },  
            colors = TopAppBarDefaults.topAppBarColors(  
                containerColor = MaterialTheme.colorScheme.primary,  
                titleContentColor = MaterialTheme.colorScheme.onPrimary,  
                navigationIconContentColor = MaterialTheme.colorScheme.onPrimary  
            ),  
            navigationIcon = {  
                IconButton(onClick = { navController?.popBackStack() }) {  
                    Icon(  
                        painter = painterResource(id = R.drawable.back_icon),  
                        contentDescription = "Back Icon"  
                    )  
                }  
            }  
        )  
    }  
)  
) { paddingValues ->
```

Gambar 20. Potongan Kode bagian TopBar

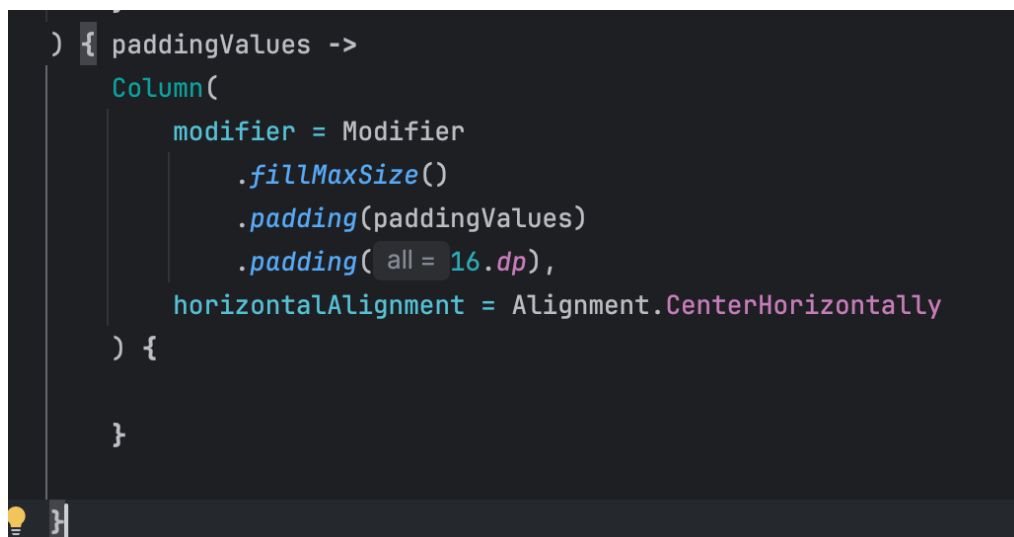
Pada saat menulis kode, terkadang kita akan menjumpai bagian kode yang berwarna merah, hal ini dapat terjadi karena kesalahan format penulisan maupun membutuhkan dependency yang perlu kita import setelah pada sesi sebelumnya kita mendeklarasikan dependency pada `build.gradle.kts`. Untuk mengetahui penyebab masalah tersebut, arahkan kursor pada kode yang berwarna merah dan tunggu beberapa saat hingga muncul suggestion seperti gambar di bawah ini kemudian pilih **"Import class 'Painter'"** :



Gambar 21. Import Suggestion

11. Membuat Column

Di dalam scaffold, silakan tambahkan column sebagai berikut:



Gambar 22. Deklarasi Column di Dalam Scaffold

12. Menambahkan Teks dan *OutlinedTextField*

Silakan tulis kode berikut di dalam scope Column.

```
Text(
    text = "Hubungi Kami",
    style = MaterialTheme.typography.titleLarge,
    modifier = Modifier.align(Alignment.Start)
)

Spacer(modifier = Modifier.height(16.dp))

OutlinedTextField(
    value = emailText,
    onChange = { emailText = it },
    label = { Text("Email Anda") },
    leadingIcon = { Icon(imageVector = Icons.Default.Email, contentDescription = "Email") },
    modifier = Modifier.fillMaxWidth(),
    shape = MaterialTheme.shapes.medium
)

Spacer(modifier = Modifier.height(16.dp))

OutlinedTextField(
    value = messageText,
    onChange = { messageText = it },
    label = { Text("Pesan") },
    modifier = Modifier.fillMaxWidth().height(120.dp),
    shape = MaterialTheme.shapes.medium
)

Spacer(modifier = Modifier.height(16.dp))

Button(
```

Gambar 23. Menambahkan Teks dan *OutlinedTextField*

Kode ini digunakan untuk Menyusun tampilan pada layar Hubungi Kami. Hal yang baru dalam praktikum ke-2 ini adalah mengimplementasikan *OutlinedTextField*. *OutlinedTextField*, merupakan komponen kotak isian modern yang memiliki garis batas di sekelilingnya. Kotak pertama dirancang khusus untuk meminta "Email Anda", yang ditambahkan ikon gambar surat di bagian dalamnya (*leadingIcon*). Setiap huruf yang diketik oleh pengguna di kotak ini akan langsung ditangkap dan disimpan ke dalam variabel *emailText*. Kemudian, kotak isian kedua disediakan untuk menampung isi "Pesan". Berbeda dengan kotak pertama, kotak pesan ini sengaja ditarik lebih tinggi (*height(120.dp)*) agar memberikan area yang cukup luas bagi pengguna untuk menuliskan kalimat panjang. Kedua kotak isian ini juga diinstruksikan untuk membentang memenuhi lebar layar ponsel (*fillMaxWidth*) dengan sudut kotak yang dibuat sedikit melengkung (*shape*).

13. Menambahkan Button

Setelah menambahkan kode pada tahapan 10, tambahkan Button sesuai seperti pada gambar 24 di bawah ini:

```
Button(  
    onClick = { },  
    modifier = Modifier.fillMaxWidth(),  
    shape = MaterialTheme.shapes.large  
) {  
    Row(  
        verticalAlignment = Alignment.CenterVertically,  
        horizontalArrangement = Arrangement.Center  
    ) {  
        Icon(imageVector = Icons.Default.Send, contentDescription = "Send")  
        Spacer(modifier = Modifier.padding(horizontal = 4.dp))  
        Text("Kirim Pesan", style = MaterialTheme.typography.labelLarge)  
    }  
}
```

Gambar 24. Source untuk Menambahkan Button

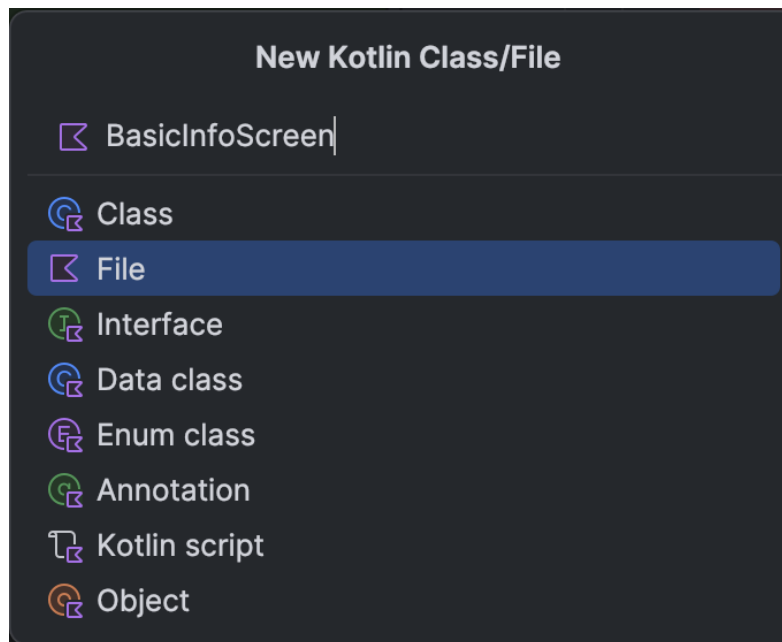
Penjelasan kode:

Kode dalam scope button tersebut digunakan untuk mengatur tampilan tombol dengan lebar hingga memenuhi lebar layar (`fillMaxWidth`). Di dalam kode tersebut kita menyusun isi tombol menggunakan `Row` yang posisinya diseimbangkan persis di tengah-tengah (`Center`) yang berisi gambar ikon pengiriman (`Icon`), serta teks "Kirim Pesan" (`Text`).

E. Modifikasi Main Activity dan Membuat Screen Baru

1. Membuat File: BasicInfoScreen.kt

Silakan tambahkan 1 file .kt dengan nama BasicInfoScreen.kt pada package ui.screen



Gambar 25. Membuat File: BasicInfoScreen.kt

2. Membuat Function BasicInfoScreen()

Silakan tambahkan function BasicInfoScreen serta deklarasi Scaffold seperti gambar berikut:

```
@OptIn( ...markerClass = ExperimentalMaterial3Api::class)
@Composable
fun BasicInfoScreen(onNavigateToContact:()->Unit){
    Scaffold(

    ) {
    }
}
```

Gambar 26. Deklarasi Function serta Scaffold

3. Memindahkan isi *function LayoutTentangJualan* dari MainActivity ke dalam Function **BasicInfoScreen**.

Silakan buka file MainActivity dan pindahkan seluruh baris kode yang terdapat pada function LayoutTentangJualan ke dalam function BasicInfoScreen.

```
@Composable
fun LayoutTentangJualan(){
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding( all = 16.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Box(
            modifier = Modifier
                .size(100.dp)
                .clip(CircleShape)
                .background(Color.Gray),
            contentAlignment = Alignment.Center
        ) {
            Text("Jualan", color = Color.White, fontWeight = FontWeight.Bold)
        }
        Spacer(modifier = Modifier.height(24.dp))

        Text(
            text = "Tentang Jualan",
            fontSize = 24.sp,
            fontWeight = FontWeight.Bold
        )
    }
}
```

Gambar 27. Kode pada LayoutTentangJualan

4. Mengganti baris kode nama produk dengan icon aplikasi

Silakan ganti kode berikut:

```
Box(  
    modifier = Modifier  
        .size(100.dp)  
        .clip(CircleShape)  
        .background(Color.Gray),  
    contentAlignment = Alignment.Center  
) {  
    Text(  
        "Jualan",  
        style = MaterialTheme.typography.headlineMedium,  
        color = MaterialTheme.colorScheme.primary  
    )  
}
```

Gambar 28. Source Code untuk Menampilkan Text nama Aplikasi

Menjadi seperti di bawah ini:

```
Image(  
    painter = painterResource(id = R.mipmap.ic_launcher_foreground),  
    contentDescription = "Logo Aplikasi",  
    modifier = Modifier.size(120.dp)  
)
```

Gambar 29. Source Code untuk Menampilkan Icon Aplikasi

5. Menambahkan Parameter untuk Menerapkan Tema Material Design

Silakan modifikasi kode yang telah dipindahkan ke BasicInfoScreen menjadi seperti gambar 30 – 31 berikut:

```

Card(
    modifier = Modifier.fillMaxWidth(),
    elevation = CardDefaults.cardElevation(defaultElevation = 2.dp),
    colors = CardDefaults.cardColors(
        containerColor = MaterialTheme.colorScheme.tertiary,
        contentColor = MaterialTheme.colorScheme.onTertiary
    )
) {
    Row(
        modifier = Modifier.padding(all = 16.dp),
        verticalAlignment = Alignment.CenterVertically
    ) {
        Text(
            text = "Misi Kami:",
            style = MaterialTheme.typography.titleLarge,
            color = Color.White,
            modifier = Modifier.weight(1f)
        )
    }
}

```

Gambar 30. Menambahkan Parameter Colors pada Card dan Text

Kemudian lanjutkan pada baris kode di bawahnya seperti pada gambar 31 berikut ini dengan menambahkan tombol.

```

Text(
    text = "Memajukan UMKM Lokal",
    modifier = Modifier.weight(2f)
)
Spacer(modifier = Modifier.weight(1f))

Button(
    onClick = onNavigateToContact,
    modifier = Modifier.fillMaxWidth().height(50.dp)
) {
    Text("Hubungi Kami", style = MaterialTheme.typography.labelLarge)
}
Spacer(modifier = Modifier.height(16.dp))

```

Gambar 31. Menambahkan Tombol pada BasicInfoScreen

Potongan kode `style = MaterialTheme.typography.titleLarge` berfungsi sebagai pengaturan tampilan yang terhubung langsung dengan konfigurasi tema di file `Theme.kt` maupun `Type.kt`. `containerColor` bertugas mengisi warna blok dasar atau latar belakang komponen tersebut dengan warna utama (`primary`) yang telah ditentukan di file tema (`Theme.kt`).

F. Membuat Halaman Navigasi

Pada bagian ini, kita akan menghubungkan 2 halaman yang telah dibuat untuk ditampilkan dalam MainActivity menggunakan NavController. Silakan buka MainActivity dan ketikkan kode berikut:

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            JualanTheme {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    val navController = rememberNavController()
                    NavHost(navController = navController, startDestination = "basic_info") {
                        composable(route = "basic_info") {
                            BasicInfoScreen(
                                onNavigateToContact = { navController.navigate(route = "form_screen") }
                            )
                        }
                        composable(route = "form_screen") {
                            HubungiKamiScreen(navController = navController)
                        }
                    }
                }
            }
        }
    }
}
```

Gambar 32. Kode untuk Navigasi pada MainActivity

Penjelasan Kode:

Potongan kode pada gambar 27 berfungsi sebagai pondasi utama sekaligus "peta jalan" saat aplikasi dijalankan. komponen Surface bertindak sebagai kanvas dasar yang dibentangkan hingga memenuhi seluruh layar (fillMaxSize) dengan warna latar standar bawaan tema. Selanjutnya, kita menciptakan "pemandu jalan" menggunakan rememberNavController() dan diatur oleh NavHost. NavHost mendeklarasikan agar yang pertama kali ditampilkan adalah "basic_info". Pada implementasi ini, kita "mendaftarkan" dua layout yaitu BasicInfoScreen yang

dibekali aksi untuk berpindah ke rute "form_screen", sedangkan yang kedua menampilkan HubungiKamiScreen.

G. Membuat Snackbar

Snackbar adalah komponen UI pada Android yang berfungsi untuk memberikan pesan atau pemberitahuan singkat di bagian bawah layar. Komponen ini dirancang untuk memberikan informasi sekilas (misalnya: "Data berhasil disimpan" atau "Pesan terkirim") tanpa mengganggu aktivitas yang sedang dilakukan oleh pengguna.

Secara singkat, ciri utama Snackbar adalah:

- **Sementara:** Pesan akan menghilang secara otomatis setelah beberapa detik.
- **Tidak Mengunci Layar:** Saat Snackbar muncul, pengguna tetap bisa menyentuh dan berinteraksi dengan bagian layar lainnya.
- **Mendukung Tombol Aksi:** Snackbar dapat disisipi tombol interaktif, seperti tombol "Batal" (*Undo*) atau "Coba Lagi" (*Retry*), yang sangat berguna jika pengguna melakukan kesalahan yang tidak disengaja.

1. Deklarasi Variabel

Silakan Kembali ke HubungiKamiScreen dan tuliskan kode berikut:

```
Usage
@OptIn( ...markerClass = ExperimentalMaterial3Api::class)
@Composable
fun HubungiKamiScreen(navController: NavController?) {
    var emailText by remember { mutableStateOf( value = "" ) }
    var messageText by remember { mutableStateOf( value = "" ) }

    val scope = rememberCoroutineScope()
    val snackbarHostState = remember { SnackbarHostState() }
```

Gambar 33. Deklarasi Variabel untuk Snackbar

Penjelasan kode:

variabel `snackbarHostState` berfungsi sebagai "manajer pengontrol" yang menyimpan antrian pesan dan mengatur kapan *Snackbar* tersebut harus tampil di layar. komponen tersebut dibungkus dengan perintah `remember` agar sistem tetap mengingat statusnya dan tidak mereset ulang pesan ketika layar sedang memperbarui tampilannya (rekomposisi). Variabel `scope` yang dihasilkan oleh `rememberCoroutineScope()` bertindak sebagai jalur eksekusi khusus di latar belakang. Karena proses memunculkan dan menahan *Snackbar* di layar selama beberapa detik merupakan tugas yang memakan waktu (*suspend function*) sehingga menerapkan proses asinkronus atau dengan kata lain, tugas tersebut tidak dapat dijalankan secara langsung pada antarmuka utama.

2. Memanggil Function `showSnackbar` pada komponen `Button`

Silakan tambahkan kode berikut pada parameter `onClick` yang terdapat pada komponen `Button`.

```
Button(  
    onClick = {  
        scope.launch {  
            snackbarHostState.showSnackbar( message = "Pesan Terkirim")  
        }  
    },  
)
```

Gambar 34. Memanggil function menampilkan *SnackBar*

Penjelasan kode:

Potongan kode di gambar 34 adalah perintah langsung untuk memunculkan pesan *SnackBar* di layar. Bagian `snackbarHostState.showSnackbar("Pesan Terkirim")` bertugas mengeksekusi dan menampilkan teks tersebut. Karena proses kemunculan pesan ini memakan waktu beberapa detik sebelum menghilang secara otomatis, perintah tersebut wajib diletakkan di dalam `scope.launch { } /` bersifat asinkronus. Pembungkus ini berfungsi mengalihkan tugas penahanan

waktu tersebut ke proses latar belakang (*coroutine*), sehingga aplikasi tidak macet (*hang*) dan layar tetap responsif saat disentuh oleh pengguna.

H. Memahami Dynamic Color

Dynamic Color adalah fitur modern pada sistem operasi Android yang secara otomatis menyesuaikan warna tampilan aplikasi mengikuti warna *wallpaper* (gambar latar) pada ponsel pengguna agar terasa lebih personal. Pada saat aplikasi dijalankan, maka tema warna belum sesuai dengan deklarasi warna yang telah kita buat. Sehingga, kita perlu mengubah nilai pada `theme.kt` di bagian **`dynamicColor: Boolean`** menjadi **`false`**.

```
@Composable
fun JualanTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    // Dynamic color is available on Android 12+
    dynamicColor: Boolean = false,
    content: @Composable () -> Unit
) {
```

Gambar 35. Konfigurasi DynamicColor pada Theme.kt

Tujuannya adalah untuk mengunci desain visual aplikasi; kita memaksa sistem agar secara konsisten hanya menggunakan warna-warna khusus yang sudah kita rancang sendiri di dalam kode. Hal ini sangat penting jika ingin mempertahankan identitas merek (*brand*) aplikasi, sehingga warna tampilannya tidak berubah-ubah dan tetap seragam meskipun pengguna mengganti-ganti *wallpaper* ponsel mereka.

Silakan jalankan aplikasi dan ubah tema perangkat Anda menjadi dark maupun light.